

4. 断线重连 - 浏览器自动处理重连 (通过 Last-Event-ID)

2.2 SSE 协议格式

SSE 消息格式：

```
代码块
1  event: message
2  data: {"type": "surfaceUpdate", "surfaceId": "main", ...}
3
4  event: done
5  data: {}
6
```

格式说明：

- event: - 事件类型 (可选, 默认 message)
- data: - 消息体 (必须是字符串)
- id: - 消息 ID (可选, 用于断点续传)
- 空行 - 表示消息结束

伪代码：SSE 连接建立

```
代码块
1  // ===== 客户端 =====
2  const eventSource = new EventSource('/a2ui/stream')
3
4  // 监听 message 事件
5  eventSource.onmessage = (event) => {
6    const a2uiMessage = JSON.parse(event.data)
7    // 处理 A2UI 消息 (传给 Parser)
8  }
9
10 // 监听自定义事件
11 eventSource.addEventListener('done', () => {
12   console.log('流式输出结束')
13 })
14
15 eventSource.addEventListener('error', (error) => {
16   console.error('SSE 错误:', error)
17 })
18
19 // ===== 服务端 =====
20 app.use(async (ctx) => {
21   if (ctx.path === '/a2ui/stream') {
22     // 设置 SSE 必需的 headers
23     ctx.set('Content-Type', 'text/event-stream')
24     ctx.set('Cache-Control', 'no-cache')
25     ctx.set('Connection', 'keep-alive')
26     ctx.set('X-Accel-Buffering', 'no') // Nginx 缓冲
27
28     // 保持连接
29     ctx.status = 200
30
31     // 后续通过 ctx.res.write() 推送消息
32   }
33 })
34
```

2.3 SSE 注意事项

1. 缓冲问题

- Nginx 反向代理会缓冲 SSE 响应, 需要关闭缓冲
- Node.js 的 response.write() 默认不缓冲, 但要注意 response.end() 会关闭连接

2. 连接数限制

- 浏览器对同一域名的 SSE 连接数有限制 (通常 6 个)
- A2UI 场景: 每个客户端一个连接, 通常够用

三、AGUI 协议 (传输层约定)